

Aaditya Dawar

1024150093

2013

## Lab Assignment 7

---

### Topics:

- A. Polymorphism
- B. Virtual Function and Pure Virtual Function
- C. Operator Overloading

- ✓ 1. Class polygon contains data member width and height and public method set\_value() to assign values to width and height. Class Rectangle and Triangle are inherited from polygon class. Both the classes contain public method calculate\_area() to calculate the area of Rectangle and Triangle. Use base class pointer to access the derived class object and show the area calculated.

### Code- ✓

```
#include <iostream>
```

```
using namespace std;
```

```
class Polygon {
```

```
protected:
```

```
    int width, height;
```

```
public:
```

```
    void set_value(int w, int h) {
```

```
        width = w;
```

```
        height = h;
```

```
    }
```

```
    virtual void calculate_area() = 0;
```

```
};
```

```
class Rectangle : public Polygon {
```

```
public:
```

```
    void calculate_area() {
```

```
        cout << "Area of Rectangle: " << width * height << endl;
```

```
    }
```

```
};
```

```
class Triangle : public Polygon {  
public:  
    void calculate_area() {  
        cout << "Area of Triangle: " << 0.5 * width * height << endl;  
    }  
};
```

```
int main() {  
    Polygon *p;  
  
    Rectangle r;  
    Triangle t;  
  
    int w, h;  
  
    cout << "Enter width and height for rectangle: ";  
    cin >> w >> h;  
    p = &r;  
    p->set_value(w, h);  
    p->calculate_area();  
  
    cout << "Enter width and height for triangle: ";  
    cin >> w >> h;  
    p = &t;  
    p->set_value(w, h);  
    p->calculate_area();  
  
    return 0;  
}
```

OUTPUT-

```
Enter width and height for rectangle: 3 17
Area of Rectangle: 51
Enter width and height for triangle: 2 10
Area of Triangle: 10

...Program finished with exit code 0
Press ENTER to exit console.[]
```

2. Write a program to create a class shape with functions *area* and *display* to find area and display the name of the shape and other essential component of the class. Create derived classes circle, rectangle and triangle each having overridden functions *area* and *display*. Write a program to find and display the area of circle, rectangle and triangle.

Code-

```
#include <iostream>
using namespace std;

class Shape {
public:
    virtual void area() = 0;
    virtual void display() = 0;
};

class Circle : public Shape {
    double r;

public:
    void input() {
        cin >> r;
    }

    void area() {
        cout << "Area: " << 3.14 * r * r << endl;
    }

    void display() {
```

```
        cout << "Shape: Circle\nRadius: " << r << endl;
    }
};
```

```
class Rectangle : public Shape {
    double l, b;
```

```
public:
```

```
    void input() {
        cin >> l >> b;
    }
```

```
    void area() {
        cout << "Area: " << l * b << endl;
    }
```

```
    void display() {
        cout << "Shape: Rectangle\nLength: " << l << " Breadth: " << b << endl;
    }
};
```

```
class Triangle : public Shape {
    double b, h;
```

```
public:
```

```
    void input() {
        cin >> b >> h;
    }
```

```
    void area() {
        cout << "Area: " << 0.5 * b * h << endl;
    }
```

```
    void display() {
        cout << "Shape: Triangle\nBase: " << b << " Height: " << h << endl;
    }
```

```
};
```

```
int main() {  
    Circle c;  
    Rectangle r;  
    Triangle t;  
  
    cout << "Enter radius: ";  
    c.input();  
    c.display();  
    c.area();  
  
    cout << "Enter length and breadth: ";  
    r.input();  
    r.display();  
    r.area();  
  
    cout << "Enter base and height: ";  
    t.input();  
    t.display();  
    t.area();  
  
    return 0;  
}
```

OUTPUT-

```

Enter radius: 7
Shape: Circle
Radius: 7
Area: 153.86
Enter length and breadth: 10 55
Shape: Rectangle
Length: 10 Breadth: 55
Area: 550
Enter base and height: 20 1
Shape: Triangle
Base: 20 Height: 1
Area: 10

...Program finished with exit code 0
Press ENTER to exit console.

```

3. Write a C++ program to compute area of right angle triangle, equilateral triangle, isosceles triangle using function overloading concept.

```
#include <iostream>
```

```
#include <cmath>
```

```
using namespace std;
```

```
double area(double base, double height) {
    return 0.5 * base * height;
}
```

```
double area(double side) {
    return (sqrt(3) / 4) * side * side;
}
```

```
double area(double a, double b, double c) {
    double s = (a + b + c) / 2;
    return sqrt(s * (s - a) * (s - b) * (s - c));
}
```

```
int main() {
    double b, h, side, a, c;

    cout << "Enter base and height (right angle triangle): ";

```

```

cin >> b >> h;
cout << "Area: " << area(b, h) << endl;

cout << "Enter side (equilateral triangle): ";
cin >> side;
cout << "Area: " << area(side) << endl;

cout << "Enter equal sides and base (isosceles triangle): ";
cin >> a >> a >> c;
cout << "Area: " << area(a, a, c) << endl;

return 0;
}

```

OUTPUT-

```

input
Enter base and height (right angle triangle): 5 10
Area: 25
Enter side (equilateral triangle): 2
Area: 1.73205
Enter equal sides and base (isosceles triangle): 2 5
5
Area: 10.8253

...Program finished with exit code 0
Press ENTER to exit console.

```

4. Write a program with Student as abstract class and create derive classes Engineering, Medicine and Science from base class Student. Create the objects of the derived classes and process them and access them using array of pointer of type base class Student.

~~#include~~ <iostream>

using namespace std;

```
class Student {
```

```
public:
```

```
    virtual void input() = 0;
```

```
    virtual void display() = 0;
```

```
};
```

```
class Engineering : public Student {
```

```
    int marks;
```

```
public:
```

```

void input() {
    cin >> marks;
}
void display() {
    cout << "Engineering Student Marks: " << marks << endl;
}
};

```

```

class Medicine : public Student {
    int marks;
public:
    void input() {
        cin >> marks;
    }
    void display() {
        cout << "Medicine Student Marks: " << marks << endl;
    }
};

```

```

class Science : public Student {
    int marks;
public:
    void input() {
        cin >> marks;
    }
    void display() {
        cout << "Science Student Marks: " << marks << endl;
    }
};

```

```

int main() {
    Student* s[3];

    Engineering e;
    Medicine m;
    Science sc;

```



```

s[0] = &e;
s[1] = &m;
s[2] = &sc;

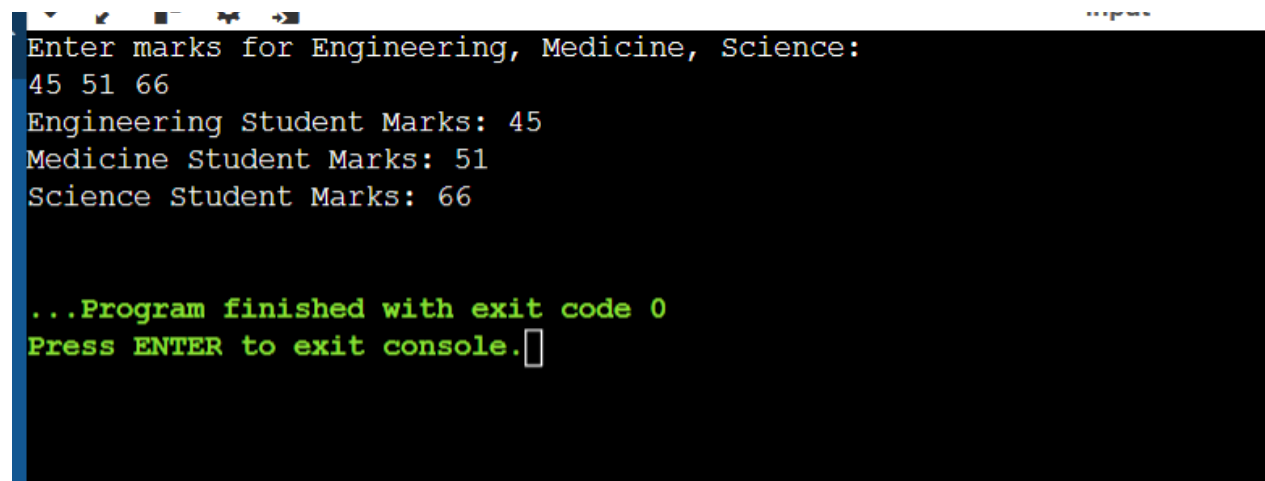
cout << "Enter marks for Engineering, Medicine, Science:\n";
for(int i = 0; i < 3; i++) {
    s[i]->input();
}

for(int i = 0; i < 3; i++) {
    s[i]->display();
}

return 0;
}

```

OUTPUT-



```

Enter marks for Engineering, Medicine, Science:
45 51 66
Engineering Student Marks: 45
Medicine Student Marks: 51
Science Student Marks: 66

...Program finished with exit code 0
Press ENTER to exit console.

```

5. Create a class Time with three private variables int h,m,s; Create a function to overload '+' operator to add two time variables.

```

int main(){
    Time t1(5,15,34),t2(9,53,58),t3;
    t3 = t1 + t2; t3.show();
}

```

```
#include <iostream>
```

```
using namespace std;
```

```
class Time {
```

```
int h, m, s;
```

```
public:
```

```
Time(int hh=0, int mm=0, int ss=0) {  
    h = hh;  
    m = mm;  
    s = ss;  
}
```

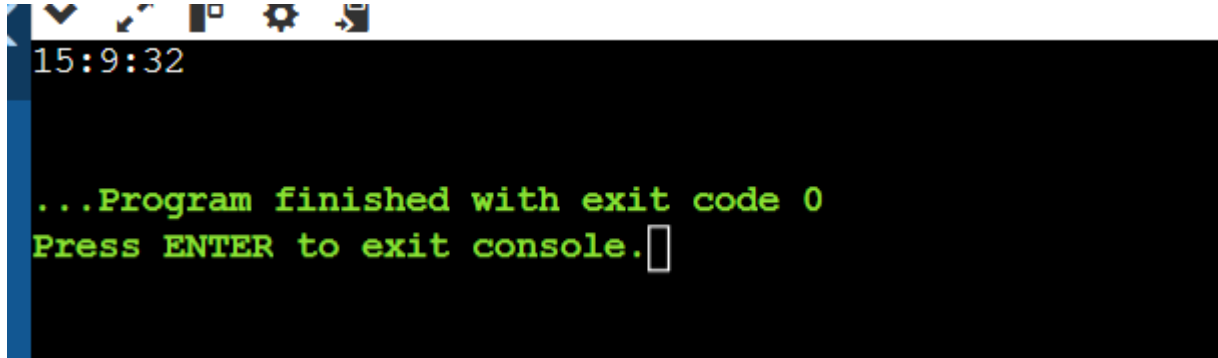
```
Time operator+(Time t) {  
    Time temp;  
  
    temp.s = s + t.s;  
    temp.m = m + t.m + temp.s / 60;  
    temp.s = temp.s % 60;  
  
    temp.h = h + t.h + temp.m / 60;  
    temp.m = temp.m % 60;  
  
    return temp;  
}
```

```
void show() {  
    cout << h << ":" << m << ":" << s << endl;  
}  
};
```

```
int main() {  
    Time t1(5,15,34), t2(9,53,58), t3;  
    t3 = t1 + t2;  
    t3.show();  
}
```

```
    return 0;
}
```

OUTPUT-

A screenshot of a Windows command prompt window. The title bar shows standard Windows icons (minimize, maximize, close) and a taskbar icon. The window has a black background with green text. The text displayed is: "15:9:32" on the first line, followed by "...Program finished with exit code 0" on the second line, and "Press ENTER to exit console." on the third line, with a cursor at the end of the third line.

6. Define a class STRING and overload == to compare two strings and + operator for concatenation of two strings.

```
#include <iostream>
#include <cstring>
using namespace std;

class STRING {
    char str[100];

public:
    void input() {
        cin >> str;
    }

    bool operator==(STRING s) {
        return strcmp(str, s.str) == 0;
    }

    STRING operator+(STRING s) {
        STRING temp;
        strcpy(temp.str, str);
        strcat(temp.str, s.str);
        return temp;
    }

    void display() {
        cout << str << endl;
    }
};

int main() {
    STRING s1, s2, s3;
```

```

cout << "Enter first string: ";
s1.input();

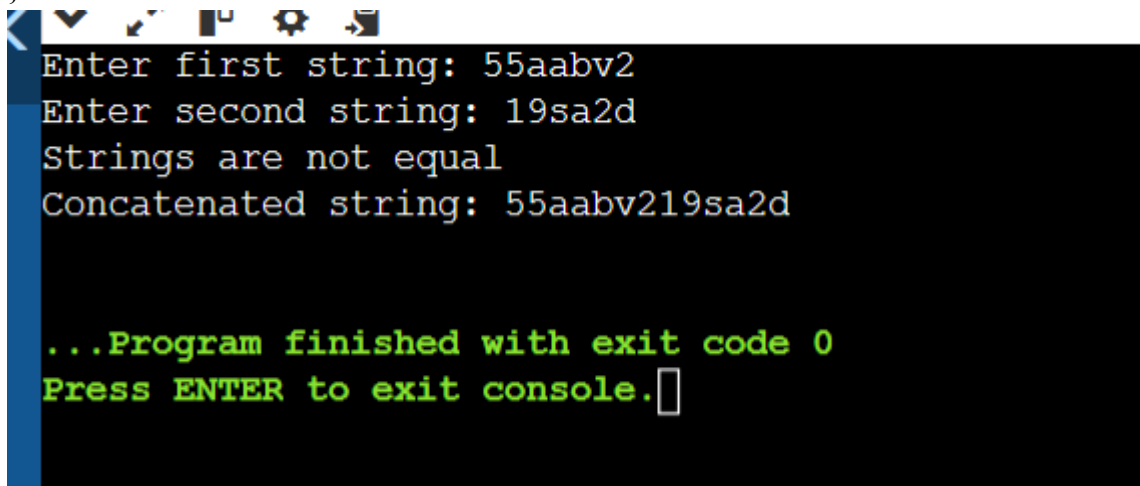
cout << "Enter second string: ";
s2.input();

if(s1 == s2)
    cout << "Strings are equal\n";
else
    cout << "Strings are not equal\n";

s3 = s1 + s2;
cout << "Concatenated string: ";
s3.display();

return 0;
}

```



```

Enter first string: 55aabv2
Enter second string: 19sa2d
Strings are not equal
Concatenated string: 55aabv219sa2d

...Program finished with exit code 0
Press ENTER to exit console.

```

7. Write a program in C++ to create a class matrix and overload \* operator using friend function to multiply two matrices.

```

#include <iostream>
using namespace std;

class Matrix {
    int a[10][10], r, c;

public:
    void input() {
        cin >> r >> c;
        for(int i = 0; i < r; i++) {
            for(int j = 0; j < c; j++) {
                cin >> a[i][j];
            }
        }
    }
}

```

```

    }

void display() {
    for(int i = 0; i < r; i++) {
        for(int j = 0; j < c; j++) {
            cout << a[i][j] << " ";
        }
        cout << endl;
    }
}

friend Matrix operator*(Matrix m1, Matrix m2);
};

Matrix operator*(Matrix m1, Matrix m2) {
    Matrix temp;
    temp.r = m1.r;
    temp.c = m2.c;

    for(int i = 0; i < temp.r; i++) {
        for(int j = 0; j < temp.c; j++) {
            temp.a[i][j] = 0;
            for(int k = 0; k < m1.c; k++) {
                temp.a[i][j] += m1.a[i][k] * m2.a[k][j];
            }
        }
    }
    return temp;
}

int main() {
    Matrix m1, m2, m3;

    cout << "Enter rows, cols and elements of first matrix:\n";
    m1.input();

    cout << "Enter rows, cols and elements of second matrix:\n";
    m2.input();

    m3 = m1 * m2;

    cout << "Resultant matrix:\n";
    m3.display();

    return 0;
}

```

OUTPUT-

```

Enter rows, cols and elements of first matrix:
2 2
1 5 6 7
Enter rows, cols and elements of second matrix:
2 2
7 1 33 6
Resultant matrix:
172 31
273 48

...Program finished with exit code 0
Press ENTER to exit console.

```

8. Overload '[]' to check array index out of bounds problem at run time.

```

#include <iostream>
using namespace std;

class Array {
    int arr[5];

public:
    void input() {
        for(int i = 0; i < 5; i++) {
            cin >> arr[i];
        }
    }

    int& operator[](int index) {
        if(index < 0 || index >= 5) {
            cout << "Index out of bounds\n";
            exit(0);
        }
        return arr[index];
    }
};

int main() {
    Array a;

    cout << "Enter 5 elements:\n";
    a.input();

    int index;
    cout << "Enter index to access: ";
    cin >> index;

    cout << "Element: " << a[index] << endl;

    return 0;
}

```

```
Enter 5 elements:
5 8 5 1 9
Enter index to access: 3
Element: 1

...Program finished with exit code 0
Press ENTER to exit console.□
```

9. Overload '()' to input arbitrary number of input arguments for an object.

```
#include <iostream>
#include <cstdarg>
using namespace std;

class Numbers {
    int sum;

public:
    Numbers() { sum = 0; }

    void operator()(int count, ...) {
        sum = 0;
        va_list args;
        va_start(args, count);

        for(int i = 0; i < count; i++) {
            sum += va_arg(args, int);
        }

        va_end(args);
    }

    void display() {
        cout << "Sum: " << sum << endl;
    }
};

int main() {
    Numbers n;

    n(3, 10, 20, 30);
    n.display();

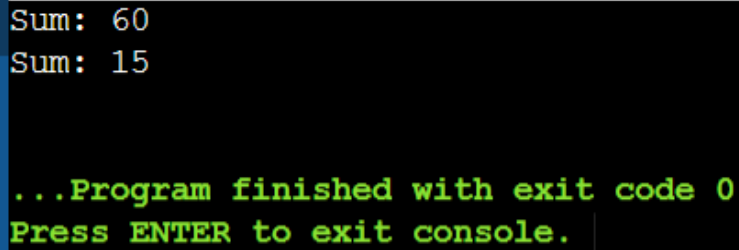
    n(5, 1, 2, 3, 4, 5);
```

```

n.display();

return 0;
}

```



```

Sum: 60
Sum: 15

...Program finished with exit code 0
Press ENTER to exit console.

```

10. Write a program in C++ to overload input operator (>>) and output operator (<<).

```

#include <iostream>
using namespace std;

class Complex {
    int real, imag;

public:
    friend istream& operator>>(istream &in, Complex &c);
    friend ostream& operator<<(ostream &out, Complex &c);
};

istream& operator>>(istream &in, Complex &c) {
    in >> c.real >> c.imag;
    return in;
}

ostream& operator<<(ostream &out, Complex &c) {
    out << c.real << " + " << c.imag << "i";
    return out;
}

int main() {
    Complex c;

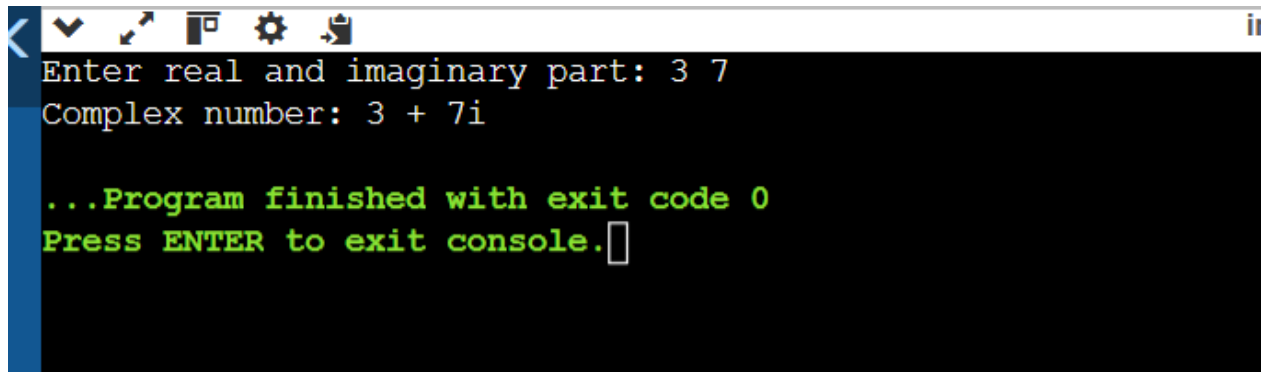
    cout << "Enter real and imaginary part: ";
    cin >> c;

    cout << "Complex number: " << c;

    return 0;
}

```





```
< ✓ ↗ 📄 ⚙️ 📋 ii
Enter real and imaginary part: 3 7
Complex number: 3 + 7i

...Program finished with exit code 0
Press ENTER to exit console. □
```

11. Write a program to convert basic data type (float) to user defined data type (object).  
class Test { private://...

public:

```
Test ( data_type) { // conversion code }  
};
```

Code-

```
#include <iostream>
```

```
using namespace std;
```

```
class Test {
```

```
    float x;
```

public:

```
    Test(float a) {
```

```
        x = a;
```

```
    }
```

```
    void display() {
```

```
        cout << "Value in object: " << x << endl;
```

```
    }
```

```
};
```

```
int main() {
```

```
    float f;
```

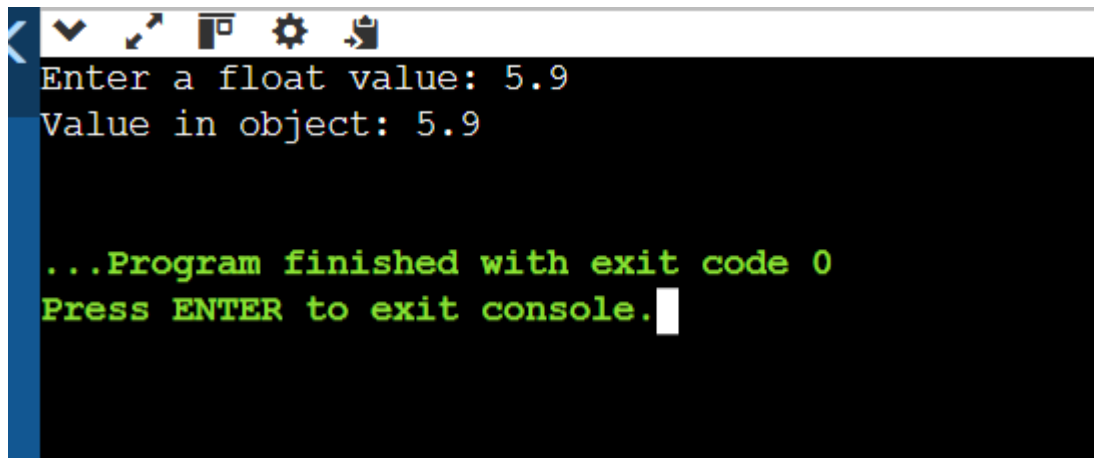
```
    cout << "Enter a float value: ";
```

```
    cin >> f;
```

```
    Test t = f;
```

```
    t.display();
```

```
return 0;  
}
```

A screenshot of a console window with a dark background. The window has a title bar with standard icons (back, forward, search, settings, and a terminal icon). The text inside the console is as follows:  
Enter a float value: 5.9  
Value in object: 5.9  
  
...Program finished with exit code 0  
Press ENTER to exit console.  
A white cursor is visible at the end of the last line.

12. Write a program to convert UDT to basic data type (float)

```
class Test { public:  
    operator data_type() { //Conversion code }  
};
```

```
#include <iostream>
```

```
using namespace std;
```

```
class Test {  
    float x;
```

```
public:
```

```
    Test(float a = 0) {  
        x = a;  
    }
```

```
    operator float() {  
        return x;  
    }
```

```
};
```

```
int main() {
```

```

Test t;

float f;

cout << "Enter a float value: ";

cin >> f;

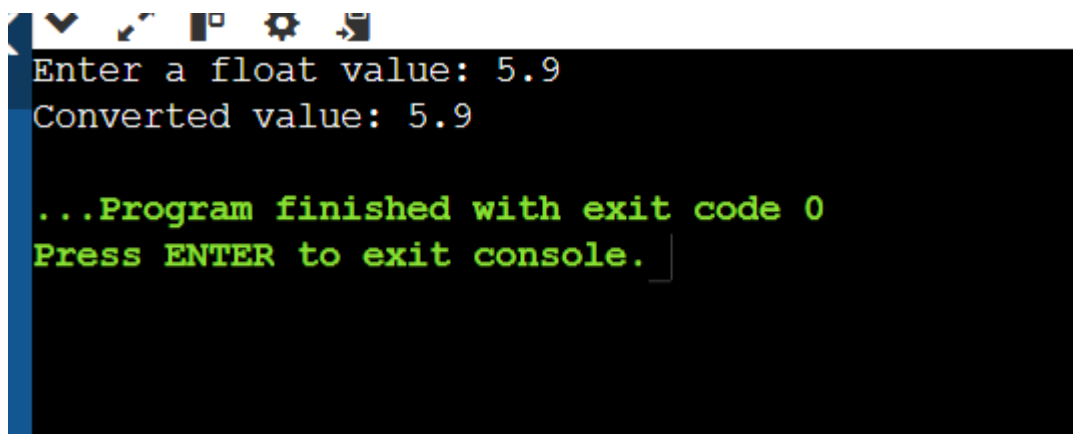
t = Test(f);

float result = t;

cout << "Converted value: " << result;

return 0;
}

```



```

Enter a float value: 5.9
Converted value: 5.9

...Program finished with exit code 0
Press ENTER to exit console.

```

13. How will you convert one UDT to another UDT. For example conversion of polar to cartesian system.

```

Polar p(10,5);
Cartesian c = p;
c.show();
#include <iostream>

#include <cmath>

using namespace std;

class Cartesian;

```

```
class Polar {  
    float r, theta;  
  
public:  
    Polar(float r1, float t1) {  
        r = r1;  
        theta = t1;  
    }  
  
    operator Cartesian();  
};
```

```
class Cartesian {  
    float x, y;  
  
public:  
    Cartesian() {  
        x = y = 0;  
    }  
  
    void set(float a, float b) {  
        x = a;  
        y = b;  
    }  
  
    void show() {  
        cout << "x = " << x << ", y = " << y << endl;  
    }  
};
```

```
};
```

```
Polar::operator Cartesian() {
```

```
    Cartesian c;
```

```
    float x = r * cos(theta);
```

```
    float y = r * sin(theta);
```

```
    c.set(x, y);
```

```
    return c;
```

```
}
```

```
int main() {
```

```
    Polar p(10, 5);
```

```
    Cartesian c;
```

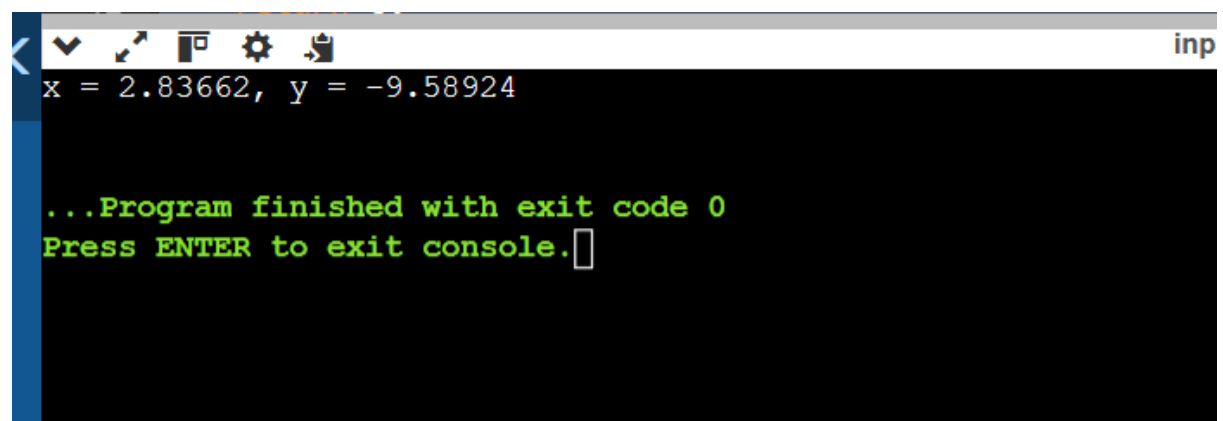
```
    c = p;
```

```
    c.show();
```

```
    return 0;
```

```
}
```

OUTPUT-



```
x = 2.83662, y = -9.58924

...Program finished with exit code 0
Press ENTER to exit console.
```